

Stack using Array

Last Updated : 13 Sep, 2025

A stack is a [linear data structure](#) that follows the [Last-In-First-Out \(LIFO\)](#) principle. It can be implemented using an array by treating the end of the array as the top of the stack.

Declaration of Stack using Array

A stack can be implemented using an array where we maintain:

- An integer array to store elements.
- A variable capacity to represent the maximum size of the stack.
- A variable top to track the index of the top element. Initially, top = -1 to indicate an empty stack.

[C++](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
class myStack {  
  
    // array to store elements  
    private int[] arr;  
  
    // maximum size of stack  
    private int capacity;  
  
    // index of top element  
    private int top;  
  
    // constructor  
    public myStack(int cap) {  
        capacity = cap;  
        arr = new int[capacity];  
        top = -1;  
    }  
}
```

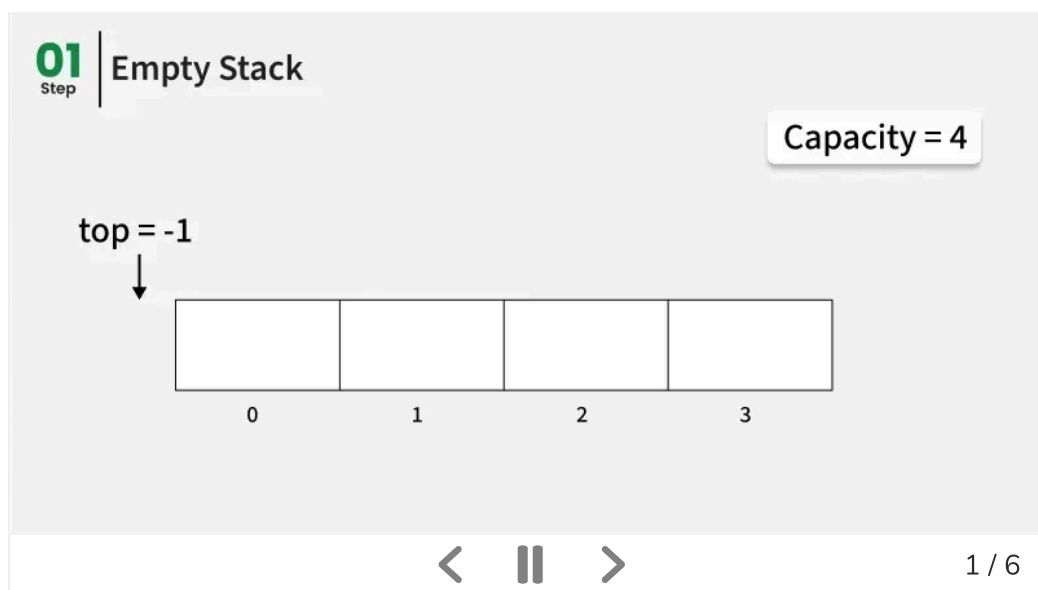


Operations On Stack

Push Operation:

Adds an item to the stack. If the stack is full, then it is said to be an **Overflow** condition.

- Before pushing the element to the stack, we check if the stack is full.
- If the stack is full ($\text{top} == \text{capacity} - 1$), then Stack Overflows and we cannot insert the element to the stack.
- Otherwise, we increment the value of top by 1 ($\text{top} = \text{top} + 1$) and the new value is inserted at top position.
- The elements can be pushed into the stack till we reach the capacity of the stack.

**C++****Java****Python****C#****JavaScript**

```
void push(int x) {  
    if (top == capacity - 1) {  
        System.out.println("Stack Overflow");  
        return;  
    }  
    arr[++top] = x;  
}
```

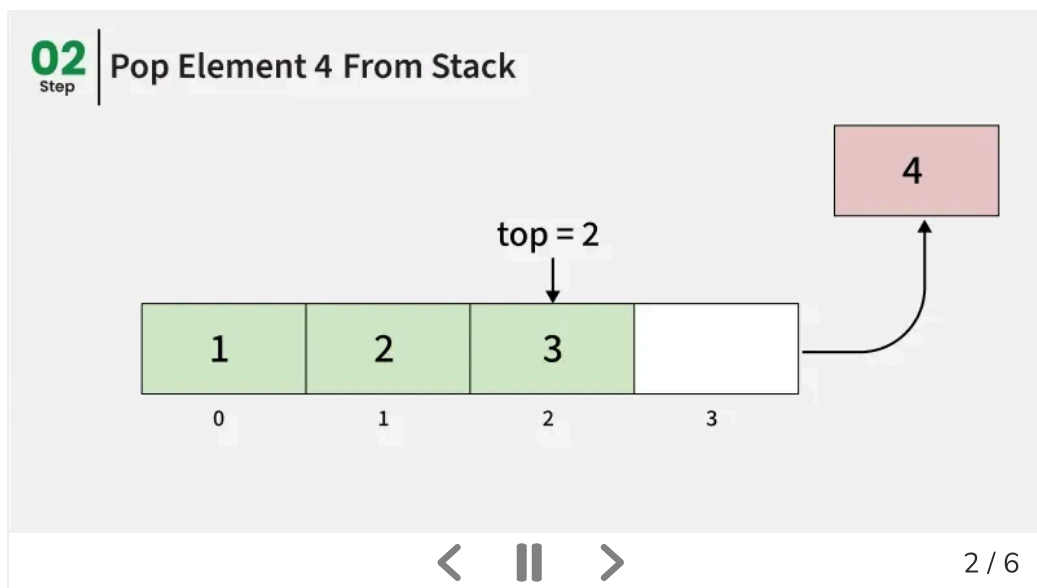
Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Pop Operation:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow** condition.

- Before popping the element from the stack, we check if the stack is empty .
- If the stack is empty ($\text{top} == -1$), then Stack Underflows and we cannot remove any element from the stack.
- Otherwise, we store the value at top, decrement the value of top by 1 ($\text{top} = \text{top} - 1$) and return the stored top value.



C++

Java

Python

C#

JavaScript

```
int pop() {  
    if (top == -1) {  
        System.out.println("Stack Underflow");  
        return -1;  
    }  
    return arr[top--];  
}
```

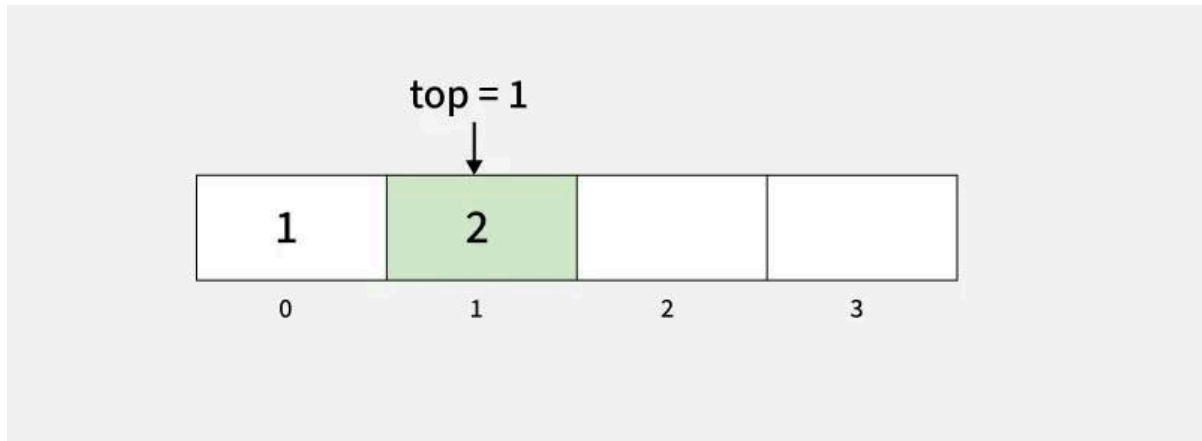
Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Top or Peek Operation in Stack:

Returns the top element of the stack.

- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty ($\text{top} == -1$), we simply print "Stack is empty".
- Otherwise, we return the element stored at **index = top**.

**C++**

Java

Python

C#

JavaScript

```
int peek() {  
    if (top == -1) {  
        cout << "Stack is Empty\n";  
        return -1;  
    }  
    return arr[top];  
}
```

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

isEmpty Operation in Stack:

Returns true if the stack is empty, else false.

- Check for the value of top in stack.
- If ($\text{top} == -1$) , then the stack is empty so return true.
- Otherwise, the stack is not empty so return false.

Empty Stack (Popping an Element will cause underflow)

top = -1



C++

Java

Python

C#

JavaScript

```
boolean isEmpty() {  
    return top == -1;  
}
```



Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

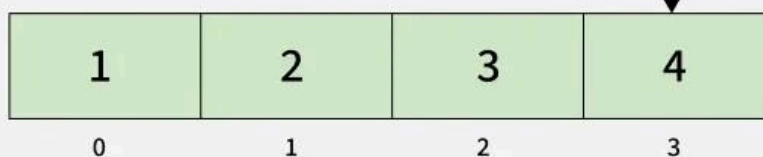
isFull Operation in Stack :

Returns true if the stack is full, else false.

- Check for the value of top in stack.
- If $(top == capacity - 1)$, then the stack is full so return true.
- Otherwise, the stack is not full so return false.

Capacity = 4

top = 3



C++

Java

Python

C#

JavaScript

```
boolean isFull() {  
  
    return top == capacity - 1;  
}
```



Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Full Implementation of Stack using Array

C++

Java

Python

C#

JavaScript

```
import java.util.Arrays;  
  
class myStack {  
  
    // array to store elements  
    private int[] arr;  
  
    // maximum size of stack  
    private int capacity;  
  
    // index of top element  
    private int top;  
  
    // constructor  
    public myStack(int cap) {  
        capacity = cap;  
        arr = new int[capacity];  
        top = -1;  
    }  
  
    // push operation  
    public void push(int x) {  
        if (top == capacity - 1) {  
            System.out.println("Stack Overflow");  
            return;  
        }  
        arr[++top] = x;  
    }  
  
    // pop operation  
    public int pop() {  
        if (top == -1) {
```



```
        System.out.println("Stack Underflow");
        return -1;
    }
    return arr[top--];
}

// peek (or top) operation
public int peek() {
    if (top == -1) {
        System.out.println("Stack is Empty");
        return -1;
    }
    return arr[top];
}

// check if stack is empty
public boolean isEmpty() {
    return top == -1;
}

// check if stack is full
public boolean isFull() {
    return top == capacity - 1;
}
}

public class Main {
    public static void main(String[] args) {
        myStack st = new myStack(4);

        // pushing elements
        st.push(1);
        st.push(2);
        st.push(3);
        st.push(4);

        // popping one element
        System.out.println("Popped: " + st.pop());

        // checking top element
        System.out.println("Top element: " + st.peek());

        // checking if stack is empty
```

```
System.out.println("Is stack empty: " +
                  (st.isEmpty() ? "Yes" : "No"));

// checking if stack is full
System.out.println("Is stack full: " +
                  (st.isFull() ? "Yes" : "No"));
    }
}
```

Output

```
Popped: 4
Top element: 3
Is stack empty: No
Is stack full: No
```

Stack Implementation using Dynamic Array

When using a fixed-size array, the stack has a maximum capacity that cannot grow beyond its initial size. To overcome this limitation, we can use **dynamic arrays**. Dynamic arrays automatically resize themselves as elements are added or removed, which makes the stack more flexible.

- In C++, we can use vector.
- In Java, we can use ArrayList.
- In Python, we can use list.
- In C#, we can use List.
- In JavaScript, we can use Array.

C++**Java****Python****C#****JavaScript**

```
import java.util.ArrayList;

class myStack {
    ArrayList<Integer> arr = new ArrayList<>();

    // push operation
    void push(int x) {
        arr.add(x);
    }
}
```

```
}

// pop operation
int pop() {
    if (arr.isEmpty()) {
        System.out.println("Stack Underflow");
        return -1;
    }
    int val = arr.get(arr.size() - 1);
    arr.remove(arr.size() - 1);
    return val;
}

// peek operation
int peek() {
    if (arr.isEmpty()) {
        System.out.println("Stack is Empty");
        return -1;
    }
    return arr.get(arr.size() - 1);
}

// check if stack is empty
boolean isEmpty() {
    return arr.isEmpty();
}

// current size
int size() {
    return arr.size();
}

public static void main(String[] args) {
    myStack st = new myStack();

    // pushing elements
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);

    // popping one element
    System.out.println("Popped: " + st.pop());
}
```

```
// checking top element
System.out.println("Top element: " + st.peek());

// checking if stack is empty
System.out.println("Is stack empty: " +
    (st.isEmpty() ? "Yes" : "No"));

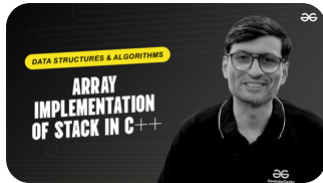
// checking current size
System.out.println("Current size: " + st.size());
    }
}
```

Output

Popped: 4
Top element: 3
Is stack empty: No
Current size: 3

Fixed Size Stack Vs Dynamic Size Stack

Features	Fixed Size	Dynamic Size
Memory Allocation	Fixed at the time of stack creation	Grows or shrinks automatically as elements change
Flexibility	Limited, cannot exceed predefined capacity	Highly flexible, no predefined size limit
Performance	Slightly faster due to no resizing overhead	May have resizing cost (amortized $O(1)$ push)
Ease of Use	Need to handle overflow/underflow manually	More convenient, no overflow (only underflow)
Use Case	Useful when maximum size is known in advance	Useful when size requirements are unknown or variable



Array
Imple
of Sta



Array
Imple
of Sta



Array Implementation of Stack in C++

[Visit Course](#)[Comment](#)[More info](#)

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem



[Array](#)
[Strings](#)
[Linked List](#)
[Stack](#)
[Queue](#)
[Tree](#)
[Graph](#)
[Searching](#)

[Sign In](#)

(201305)

Registered Address:

K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305



Company

[About Us](#)
[Legal](#)
[Privacy Policy](#)
[Contact Us](#)
[Advertise with us](#)
[GFG Corporate Solution](#)
[Campus Training Program](#)

Explore

[POTD](#)
[Job-A-Thon](#)
[Community](#)
[Blogs](#)
[Nation Skill Up](#)

Tutorials

[Programming Languages](#)
[DSA](#)
[Web Technology](#)
[AI, ML & Data Science](#)
[DevOps](#)
[CS Core Subjects](#)
[Interview Preparation](#)
[GATE](#)
[Software and Tools](#)

Courses

[IBM Certification](#)
[DSA and Placements](#)
[Web Development](#)
[Programming Languages](#)
[DevOps & Cloud](#)
[GATE](#)
[Trending Technologies](#)

Videos

[DSA](#)
[Python](#)
[Java](#)
[C++](#)
[Web Development](#)
[Data Science](#)
[CS Subjects](#)

Preparation Corner

[Aptitude](#)
[Puzzles](#)
[GfG 160](#)
[DSA 360](#)
[System Design](#)

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved